

NoteVision AI: Handwritten Note Digitization Using an EMNIST CNN and Vision Transformer Pipeline

Sneha Bomma
*dept. of Computer Science and
Engineering
Koneru Lakshmaiah Education
Foundation
Hyderabad, India
2420030231@klh.edu.in*

Kola Hitheshini
*dept. of Computer Science and
Engineering
Koneru Lakshmaiah Education
Foundation
Hyderabad, India
2420030233@klh.edu.in*

Lalitha Mahisri Mallepudi
*dept. of Computer Science and
Engineering
Koneru Lakshmaiah Education
Foundation
Hyderabad, India
2420030445@klh.edu.in*

Abstract—Handwritten text recognition bridges physical records and digital workflows, but natural handwriting remains notoriously difficult to parse due to inconsistent stroke shapes, irregular spacing, and diverse writing styles. While benchmark datasets like EMNIST provide an effective baseline for isolated character recognition, they falter when processing full handwritten documents. This paper presents NoteVision AI, an end-to-end digitization framework combining an EMNIST-trained Convolutional Neural Network (CNN) with Microsoft TrOCR. In this pipeline, the CNN evaluates individual character strokes, geometry, and classification confidence on segmented regions, while TrOCR transcribes complete text lines sequentially to preserve semantic context and word coherence. The system processes raw document uploads through an integrated pipeline of image preprocessing, contour-based segmentation, dual-model inference, human-in-the-loop confidence verification, and searchable PDF compilation.

Keywords—*Handwritten Text Recognition, EMNIST, Convolutional Neural Networks, Vision Transformer, TrOCR, Document Digitization.*

I. INTRODUCTION

Physical notebooks and handwritten logs remain indispensable in academic and professional settings. However, paper records cannot be queried, edited, or indexed efficiently. While modern optical character recognition (OCR) engines reliably parse uniform machine-printed text, handwritten pages introduce high variability. Writers employ distinct letterforms, alternate between cursive and print, vary their pen pressure, and write across unaligned lines under irregular lighting conditions.

Convolutional Neural Networks excel at extracting spatial features from small image patches, making standard collections like the Extended MNIST (EMNIST) dataset effective for character classification benchmarks [1]. Despite this capability, natural notes present continuous words, overlapping ascenders and descenders, and unpredictable word gaps rather than neatly isolated glyphs. Evaluating each character in isolation causes frequent word-level errors.

To overcome these structural limitations while retaining character-level diagnostics, NoteVision AI pairs a custom

EMNIST CNN with Microsoft’s Transformer-based OCR architecture (microsoft/trocr-base-handwritten) [6]. The CNN isolates character-level visual patterns and highlights ambiguous glyphs, while TrOCR processes horizontal text strips as coherent sequences. Wrapped in a modular browser interface, NoteVision AI transforms raw handwritten uploads into editable, exportable digital documents.

II. RELATED WORK

Cohen et al. introduced EMNIST to extend the standard MNIST format into alphanumeric recognition, extracting 28×28 pixel grayscale crops from NIST Special Database 19 [1]. The dataset includes multiple configurations—such as ByClass, ByMerge, Balanced, and Letters—enabling fine-grained tuning across digit and letter categories [2].

Chooi and Ghafar demonstrated that CNN architectures trained on EMNIST learn stroke representations directly from raw pixels, achieving high character-classification accuracy with minimal manual feature design [3]. Nevertheless, document-level transcription requires understanding layout and reading order. As Jo et al. observed, unconstrained text segmentation remains an error-prone bottleneck; connected strokes and irregular letter spacing frequently cause adjacent letters to merge or single glyphs to fracture [4].

To address these spatial dependencies, modern OCR systems employ sequence-to-sequence vision architectures. Microsoft TrOCR dispenses with traditional convolutional backends and external language decoders by coupling an image Transformer encoder with an autoregressive text Transformer decoder [6], [7]. Pre-trained on extensive handwritten collections like the IAM dataset, TrOCR reads full line crops natively. NoteVision AI incorporates both paradigms: OpenCV and the EMNIST CNN supply localized visual bounding and confidence metrics, while TrOCR transcribes full text lines across the page.

III. SYSTEM ARCHITECTURE

The NoteVision AI architecture consists of seven sequential modules:

A. Image Preprocessing

Mobile and document captures often suffer from page warping, shadows, and low contrast. Using OpenCV, raw inputs undergo grayscale conversion, Gaussian noise filtering, contrast normalization, and adaptive binarization to separate ink strokes from paper fibers.

B. Segmentation & Line Grouping

Morphological filtering and contour detection identify active ink regions. Bounding boxes are filtered by area and sorted along vertical and horizontal axes to reconstruct lines in natural reading order without interleaving text across adjacent rows [4].

C. EMNIST-Based CNN Classification

Detected character patches are centered, padded, and normalized to 28×28 dimensions to match the CNN's input layer. The network outputs softmax probability distributions, recording the primary predicted label alongside top-ranked alternatives.

D. TrOCR Sequence Recognition

Simultaneously, segmented horizontal line strips pass into the TrOCR pipeline, where the Transformer decoder generates full-sentence text sequences informed by surrounding linguistic context [7].

E. Alignment & Confidence Review

The system cross-references character-level CNN confidence against the generated TrOCR lines. Ambiguous or low-probability detections (such as confusions between 'O' and '0', or 'l' and '1') are flagged for user inspection.

F. Human-in-the-Loop Verification

Using a Streamlit workspace, users inspect the extracted transcript alongside the original image, rectifying transcription errors directly in a side-by-side text editor.

G. Document Generation

The verified, corrected transcript compiles into a standardized, searchable PDF document ready for archiving or export.

IV. METHODOLOGY AND IMPLEMENTATION

A. Data Preparation & CNN Training

The character classifier is trained using standard EMNIST splits. Preprocessing accounts for the dataset's native rotation and transposition anomalies [2]. The architecture applies stacked convolutional and pooling stages to extract edge and stroke primitives before feeding fully connected classification layers.

B. TrOCR Integration

The application loads the microsoft/tocr-base-handwritten weights via Hugging Face Transformers [6]. Processing whole-line crops allows the model's self-attention layers to infer ambiguous strokes from semantic context [7].

C. Modular Software Design

The platform separates core functionalities: image manipulation runs through OpenCV, character classification through TensorFlow, line transcription through Hugging Face, and user interaction through Streamlit. This modular structure allows independent model fine-tuning and segmentation updates without refactoring the front-end interface.

V. PRACTICAL CHALLENGES AND DISCUSSION

Real-world deployment revealed several key obstacles:

A. The Domain Gap

Models trained strictly on isolated character sets struggle with freehand writing. Writers vary spacing, curve margins, and connect adjacent letters. Applying character-by-character prediction in isolation frequently leads to missed spaces and garbled words.

B. Segmentation Vulnerabilities

Irregular handwriting styles frequently trigger contour fragmentation, splitting characters with gaps into separate crops or fusing closely spaced glyphs into single contours [4]. TrOCR mitigates this vulnerability by decoding complete visual lines without requiring discrete character boundaries.

C. Character Ambiguities

Without contextual cues, distinct characters can appear visually identical (e.g., uppercase 'S' and digit '5', or lowercase 'c' and 'e'). NoteVision AI addresses this through an editable interface: low-confidence segments are brought to the user's attention for inline correction prior to export.

VI. CONCLUSION AND FUTURE WORK

NoteVision AI demonstrates that pairing localized character-level CNN evaluation with line-level vision Transformers yields a resilient workflow for handwritten document digitization. By coupling automated transcription with an interactive human-in-the-loop review interface, the platform ensures clean digital outputs while accommodating real-world handwriting variability.

Future extensions will explore fine-tuning TrOCR on specialized domain lexicons (e.g., medical and engineering notes), implementing automated multi-column layout detection, and parsing mathematical expressions and tables into structured Markdown and LaTeX.

REFERENCES

- [1] G. Cohen, S. Afshar, J. Tapson, and A. van Schaik, "EMNIST: an extension of MNIST to handwritten letters," in Proc. Int. Joint Conf. Neural Netw. (IJCNN), 2017. doi: 10.1109/IJCNN.2017.7966217.
- [2] TensorFlow Datasets, "EMNIST Dataset." [Online]. Available: <https://www.tensorflow.org/datasets/catalog/emnist>
- [3] S. L. Chooi and A. S. B. Ab Ghafar, "Handwritten Character Recognition Using Convolutional Neural Network," Prog. Eng. Appl. Technol., 2021.
- [4] J. Jo, H. I. Koo, J. W. Soh, and N. I. Cho, "Handwritten Text Segmentation via End-to-End Learning of Convolutional Neural Network," arXiv:1906.05229, 2019.

- [5] TensorFlow, “EMNIST character recognition task documentation.” [Online]. Available: https://www.tensorflow.org/federated/api_docs/python/tff/simulation/baselines/emnist/create_character_recognition_task
- [6] Hugging Face, “microsoft/trocr-base-handwritten Model Card.” [Online]. Available: <https://huggingface.co/microsoft/trocr-base-handwritten>
- [7] M. Li, T. Lv, J. Chen, L. Cui, Y. Lu, D. Florencio, C. Zhang, Z. Li, and F. Wei, “TrOCR: Transformer-based Optical Character Recognition with Pre-trained Models,” arXiv:2109.10282, 2021.